Question 1

List the customers who have never placed an order.

```
SELECT sc.customer_id, sc.first_name, sc.last_name, sc.city
FROM sales.customers AS sc
LEFT JOIN sales.orders AS so ON sc.customer_id = so.customer_id
WHERE so.order_id IS NULL
ORDER BY sc.last_name ASC, sc.first_name ASC;
```

Question 2

List only the customers who have placed at least 3 orders, along with their order counts.

```
SELECT
    sc.customer_id,
    sc.first_name,
    COUNT(so.order_id) AS [Order Count]
FROM sales.customers AS sc
LEFT JOIN sales.orders AS so ON sc.customer_id = so.customer_id
GROUP BY sc.customer_id, sc.first_name
HAVING COUNT(so.order_id) >= 3
ORDER BY [Order Count] DESC;
```

Question 3

Considering only products priced above 1,000, list the brands that have at least 5 products.

```
SELECT
    pb.brand_id,
    pb.brand_name,
    COUNT(pp.product_id) AS ProductCount
FROM production.products AS pp
INNER JOIN production.brands AS pb
    ON pp.brand_id = pb.brand_id
WHERE pp.list_price > 1000
GROUP BY pb.brand_id, pb.brand_name
HAVING COUNT(pp.product_id) >= 5
ORDER BY ProductCount ASC;
```

Question 4

Considering only 2018 model products, list the brands whose average product price is above 1,500.

```
SELECT
    pb.brand_id,
    pb.brand_name,
    COUNT(pp.product_id) AS [Product Count],
    AVG(pp.list_price) AS [Average Price]
FROM production.products AS pp
INNER JOIN production.brands AS pb
    ON pp.brand_id = pb.brand_id
WHERE pp.model_year = 2018
GROUP BY pb.brand_id, pb.brand_name
HAVING AVG(pp.list_price) > 1500
ORDER BY [Average Price] DESC;
```

Question 5

Considering only products with a list price above 500, list the categories that have at least 10 products.

```
SELECT
    pc.category_id,
    pc.category_name,
    COUNT(pp.product_id) AS [Product Count],
    AVG(pp.list_price) AS [Average Price]
FROM production.products AS pp
INNER JOIN production.categories AS pc
    ON pp.category_id = pc.category_id
WHERE pp.list_price > 500
GROUP BY pc.category_id, pc.category_name
HAVING COUNT(pp.product_id) >= 10
ORDER BY [Product Count] ASC;
```

Question 6

Considering only orders placed in 2018, list the customers who have at least 3 different orders and whose total spending after discounts is above 10,000. The result should include the customer ID, first name, last name, order count and total spending. Sort by total spending in descending order.

```
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    COUNT(DISTINCT o.order_id) AS [Order Count],
    SUM(oi.quantity * oi.list_price * (1 - oi.discount)) AS [Total Spending]
FROM sales.order_items AS oi
INNER JOIN sales.orders AS o
    ON oi.order_id = o.order_id
INNER JOIN sales.customers AS c
    ON o.customer_id = c.customer_id
WHERE o.order_date >= '20180101' AND o.order_date < '20190101'
GROUP BY c.customer_id, c.first_name, c.last_name
HAVING COUNT(DISTINCT o.order_id) >= 3
    AND SUM(oi.quantity * oi.list_price * (1 - oi.discount)) > 10000
ORDER BY [Total Spending] DESC;
```

Question 7

Considering only orders placed in 2017, list the customers who ordered a total of 20 or more product units. The result should include the customer ID, first name, last name, number of distinct orders and total quantity of products ordered. Sort by total quantity in descending order.

```
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    COUNT(DISTINCT o.order_id) AS [Order Count],
    SUM(oi.quantity) AS TotalQuantity
FROM sales.order_items AS oi
INNER JOIN sales.orders AS o
    ON oi.order_id = o.order_id
INNER JOIN sales.customers AS c
    ON o.customer_id = c.customer_id
WHERE o.order_date >= '20170101' AND o.order_date < '20180101'
GROUP BY c.customer_id, c.first_name, c.last_name
HAVING SUM(oi.quantity) >= 20
ORDER BY TotalQuantity DESC;
```

Question 8

List each employee's first and last name together with their manager's first and last name. Employees without a manager should also appear in the result. Use only the sales.staffs table.

Output columns:

- Employee ID
- Employee first and last name
- Manager first and last name

```
SELECT
    employee.staff_id AS [Employee ID],
    employee.first_name AS [Employee First Name],
    employee.last_name AS [Employee Last Name],
    manager.staff_id AS [Manager ID],
    manager.first_name AS [Manager First Name],
    manager.last_name AS [Manager Last Name]
FROM sales.staffs AS employee
LEFT JOIN sales.staffs AS manager
    ON employee.manager_id = manager.staff_id
ORDER BY employee.staff_id;
```

Question 9

Find pairs of coworkers who work in the same store. The two employees must meet the following conditions:

- They work in the same store.
- They report to the same manager, and the manager is not NULL.
- Both are active (active = 1).

The result should include the store name and the first and last names of both employees. An employee must not be paired with themselves. Also, since A–B and B–A are the same pair, only one of them should be shown.

```
SELECT
    s.store_name AS [Store Name],
    [first employee].first_name AS [Employee 1 First Name],
    [first employee].last_name AS [Employee 1 Last Name],
    [second employee].first_name AS [Employee 2 First Name],
    [second employee].last_name AS [Employee 2 Last Name]
FROM sales.stores AS s
INNER JOIN sales.staffs AS [first employee]
    ON s.store_id = [first employee].store_id
INNER JOIN sales.staffs AS [second employee]
    ON s.store_id = [second employee].store_id
WHERE [first employee].store_id = [second employee].store_id
    AND [first employee].manager_id IS NOT NULL
    AND [second employee].manager_id IS NOT NULL
    AND [first employee].manager_id = [second employee].manager_id
    AND [first employee].active = 1
    AND [second employee].active = 1
    AND [first employee].staff_id < [second employee].staff_id;
```

SQL Query Exercises 1

The queries in this document were written in Microsoft SQL Server (T-SQL) using the BikeStores sample database.

Mert Zeki Öztürk